

BẢN IN KỊCH BẢN HẠ TẦNG TERRAFORM IAAC:

main.tf

Hệ thống Quản lý và Số hóa Tài liệu Thư viện HCMUS (HCMUS-LDMS)

THÔNG TIN TẬP NGUỒN (FILE CONTROL)

Trường thông tin	Nội dung chi tiết
Tên tệp tin (File Name):	main.tf
Đường dẫn trong Repo (Path):	terraform/main.tf
Công nghệ:	HashiCorp Terraform (Infrastructure as Code - IaC)
Mô tả chức năng:	Kịch bản khởi tạo và cấu hình toàn bộ tài nguyên hạ tầng (Networks, Volumes, Containers)

1. Toàn văn mã nguồn kịch bản `terraform/main.tf`

```
# =====
# Terraform Main Configuration - HCMUS-LDMS Infrastructure as Code (IaaS)
# =====

terraform {
  required_version = "≥ 1.5.0"
  required_providers {
    docker = {
      source = "kreuzwerker/docker"
      version = "~> 3.0"
    }
  }
}

provider "docker" {}

# --- Isolated Bridge Network ---

resource "docker_network" "ldms_network" {
  name = "${var.project_name}_network"
  driver = "bridge"
}

# --- Persistent Storage Volumes ---

resource "docker_volume" "postgres_data" {
  name = "${var.project_name}_postgres_data"
}

resource "docker_volume" "minio_data" {
  name = "${var.project_name}_minio_data"
}

resource "docker_volume" "grafana_data" {
  name = "${var.project_name}_grafana_data"
}

# --- Container Images ---

resource "docker_image" "postgres" {
  name = "postgres:16"
  keep_locally = true
}

resource "docker_image" "minio" {
  name = "minio/minio:latest"
  keep_locally = true
}

resource "docker_image" "mailhog" {
  name = "mailhog/mailhog:latest"
  keep_locally = true
}
```

```

resource "docker_image" "prometheus" {
  name          = "prom/prometheus:latest"
  keep_locally = true
}

resource "docker_image" "grafana" {
  name          = "grafana/grafana:latest"
  keep_locally = true
}

# --- Infrastructure Services ---

# 1. PostgreSQL Database
resource "docker_container" "postgres" {
  name = "${var.project_name}_postgres"
  image = docker_image.postgres.image_id

  networks_advanced {
    name = docker_network.ldms_network.name
  }

  env = [
    "POSTGRES_USER=${var.postgres_user}",
    "POSTGRES_PASSWORD=${var.postgres_password}",
    "POSTGRES_DB=${var.postgres_db}"
  ]

  ports {
    internal = 5432
    external = var.postgres_port
  }

  volumes {
    volume_name      = docker_volume.postgres_data.name
    container_path = "/var/lib/postgresql/data"
  }

  restart = "unless-stopped"
}

# 2. MinIO S3 Object Storage
resource "docker_container" "minio" {
  name      = "${var.project_name}_minio"
  image     = docker_image.minio.image_id
  command = ["server", "/data", "--console-address", ":9001"]

  networks_advanced {
    name = docker_network.ldms_network.name
  }

  env = [
    "MINIO_ROOT_USER=${var.minio_root_user}",
    "MINIO_ROOT_PASSWORD=${var.minio_root_password}"
  ]

  ports {
    internal = 9000
  }

```

```

    external = var.minio_api_port
  }

  ports {
    internal = 9001
    external = var.minio_console_port
  }

  volumes {
    volume_name    = docker_volume.minio_data.name
    container_path = "/data"
  }

  restart = "unless-stopped"
}

# 3. MailHog SMTP Mock Service
resource "docker_container" "mailhog" {
  name = "${var.project_name}_mailhog"
  image = docker_image.mailhog.image_id

  networks_advanced {
    name = docker_network.ldms_network.name
  }

  ports {
    internal = 1025
    external = 1025
  }

  ports {
    internal = 8025
    external = var.mailhog_ui_port
  }

  restart = "unless-stopped"
}

# 4. Prometheus Metrics Monitoring
resource "docker_container" "prometheus" {
  name = "${var.project_name}_prometheus"
  image = docker_image.prometheus.image_id

  networks_advanced {
    name = docker_network.ldms_network.name
  }

  ports {
    internal = 9090
    external = var.prometheus_port
  }

  restart = "unless-stopped"
}

# 5. Grafana Monitoring Dashboard
resource "docker_container" "grafana" {

```

```
name = "${var.project_name}_grafana"
image = docker_image.grafana.image_id

networks_advanced {
  name = docker_network.ldms_network.name
}

env = [
  "GF_SECURITY_ADMIN_USER=admin",
  "GF_SECURITY_ADMIN_PASSWORD=admin",
  "GF_USERS_ALLOW_SIGN_UP=false"
]

ports {
  internal = 3000
  external = var.grafana_port
}

volumes {
  volume_name      = docker_volume.grafana_data.name
  container_path = "/var/lib/grafana"
}

restart = "unless-stopped"
}
```